SymbiYosys coursework exercises

John Wickerson

Autumn term 2025

This year's SymbiYosys coursework concerns the verification of a 5-bit unsigned integer divider.

Marking principles. If you have completed a task in full, you will get full marks for it and it is not necessary to show your working. If you have not managed to complete a task, partial credit may be given if you can demonstrate your thought process.

Submission process. You are expected to produce a single directory inside the Github repository I have made for your pair. This directory should contain your solutions to all of the tasks below that you have attempted. You may include `.sv` files and `.sby` files in your directory. You are welcome to show your working on incomplete tasks by decorating your file with `/*comments*/` or `//comments`.

Plagiarism policy. You **are** allowed to consult the coursework tasks from previous years – the questions and model solutions for these are available. You **are** allowed to consult internet sources like SymbiYosys and SystemVerilog tutorials. You **are** allowed to work together with the other student in your pair. You **are** allowed to ask questions on Stack Overflow or the Yosys mailing list, but make your questions generic (e.g. "Why am I getting a syntax error when I use the |=> operator?"); please **don't** ask for solutions to these specific tasks! Please **don't** share your answers to these tasks outside of your own pair, but please **do** help each other with general SymbiYosys tips and tricks – after all, when one student helps another student, *both* learn!

[0] Revision history: first version, 27 November 2025.

1 Verifying a 5-bit divider

Here is a Verilog design for dividing two 5-bit unsigned integers, `num` and `den`. It produces a 5-bit quotient (`quo`) and a 5-bit remainder (`rem`). (The code is also available in Github.)

```
module divider (           // 5-bit unsigned integer divider
  input            clk,    // clock
  input            rst,    // reset
  input            go,     // start calculation
  input [4:0]      num,    // input: numerator
  input [4:0]      den,    // input: denominator
  output [4:0]     rem,    // result: remainder
  output reg [4:0] quo,    // result: quotient
  output reg       busy,   // calculation in progress
  output reg       done,   // calculation is complete
  output reg       valid,  // result is valid
  output reg       dbz     // divide-by-zero detected!
);

  reg [4:0] den_latched;  // copy of denominator
  reg [5:0] acc;          // accumulator
  reg [2:0] ctr;          // iteration counter

  initial begin
    busy <= 0;
    done <= 0;
    valid <= 0;
    dbz <= 0;
    quo <= 0;
    den_latched <= 0;
    ctr <= 0;
  end

  assign rem = acc[5:1];

  always @(posedge clk) begin
    if (rst) begin
      busy <= 0;
      done <= 0;
      valid <= 0;
      dbz <= 0;
      quo <= 0;
      den_latched <= 0;
      ctr <= 0;
    end else if (go) begin
      valid <= 0;
      ctr <= 0;
      if (den == 5'b00000) begin
```

```
        busy <= 0;
        done <= 1;
        dbz <= 1;
      end else begin
        done <= 0;
        busy <= 1;
        dbz <= 0;
        den_latched <= den;
        {acc, quo} <= {5'b0, num, 1'b0};
      end
    end else begin
      if (ctr == 4) begin
        ctr <= 0;
        busy <= 0;
        done <= 1;
        valid <= 1;
      end else begin
        done <= 0;
        ctr <= ctr + 1;
      end
      if (acc >= den_latched)
        {acc, quo} <= {acc - den_latched, quo, 1'b1};
      else
        {acc, quo} <= {acc, quo, 1'b0};
    end
  end
`ifdef FORMAL
`ifdef VERIFIC
  // Verification goes here.
`endif
`endif
endmodule
```

Use SymbiYosys to prove the following properties about the design. Most of the properties are phrased in a slightly vague or slightly incorrect way. Part of your job as a verification engineer is to make these properties *precise* and *correct*.

1. If `go` goes high and the denominator is nonzero, then in the next clock cycle, `busy` will be high and `valid` will be low.

2. If `go` goes high and the denominator is nonzero, then for the next five clock cycles, `busy` will be high and `valid` will be low.

3. If `go` goes high and the denominator is nonzero, then six clock cycles later, `busy` will be low and `valid` will be high.

4. If `go` goes high and the denominator is nonzero, then six clock cycles later, the equation $\texttt{num} = \texttt{quo} \times \texttt{den} + \texttt{rem}$ will hold.

5. If `go` goes high and the denominator is nonzero, then `den_latched` will latch the value of `den`.

6. If `go` goes high and the denominator is zero, then `done` and `dbz` will both go high.

7. The `done` signal never stays high for more than one clock cycle (except when `dbz` is set).

8. The produced remainder will always be smaller than the given denominator.

9. The produced values of `quo` and `rem` will always be in the range $[0..31]$, but not *all* values in that range are actually producible – for instance, no input values for `num` and `den` will ever result in $\texttt{quo} = 6$ and $\texttt{rem} = 4$. Write down a condition on `quo` and `rem` that captures exactly the producible values, and prove that all produced values of `quo` and `rem` satisfy this condition.

10. Following on from the above: use cover statements to prove that all the values of `quo` and `rem` that satisfy your condition are actually producible. (It is possible to achieve this by writing a *long* list of cover statements. But can you manage it more succinctly with a *single* cover statement? Verilog's `for`-loop may be helpful here.)

11. Devise and prove a 'loop invariant' for the divider. That is, devise a condition on the intermediate values of `acc` and `quo` that holds on each of the six steps of the computation.

Here are a few hints and tips.

- You can assume that we are only interested in assertions that hold at rising clock edges, such as

```
always @(posedge clk) assert property (foo);
```

- If it helps for any of the tasks, you can add extra registers to your design that you use during the proof, as long as these registers only appear inside the

```
`ifdef FORMAL ... `endif
```

block so that they cannot affect synthesis.

- When phrasing some of these properties, you may find a couple of additional SystemVerilog constructions helpful. For instance,

  ```
  a ##1 b |=> c ##1 d
  ```

  expresses that if `a` holds in clock cycle n and `b` holds in cycle $n+1$, then `c` and `d` will hold in cycles $n+2$ and $n+3$ respectively. And

  ```
  a ##1 b[*2:4] |-> c
  ```

  expresses that if `a` holds in one clock cycle and then `b` holds for the next two, three, or four consecutive clock cycles, then at that point `c` will hold.